

SAE 1.01 · BUT1 · ANNÉE 2026-2027

SAE 1.01 — Gestion d'une bibliothèque de prêt

Guide de l'étudiant : déroulement, attendus, évaluation et conseils méthodologiques

Python · Fonctions · Listes · Dictionnaires · JSON · Tests unitaires · Notebook Jupyter

OBJECTIFS

- Découper un problème réel en **fonctions** simples, documentées et réutilisables.
- Modéliser des données avec des **listes**, puis migrer vers des **dictionnaires** et le format **JSON**.
- Écrire et exécuter des **tests unitaires** pour chaque fonction, d'abord fournis puis rédigés par vous-mêmes.
- Assembler les fonctions dans un **programme principal** interactif et robuste.
- Être capable d'**expliquer et défendre** chaque ligne de votre code, à l'oral comme à l'écrit, sans ordinateur.

1. Présentation de la SAE

Vous allez développer, **en binôme**, une application Python de gestion d'une bibliothèque de prêt. L'application gère deux ensembles d'informations :

- le **catalogue** : les livres que possède la bibliothèque (ajout, suppression, recherche, enregistrement dans un fichier) ;
- les **emprunts** : quels lecteurs ont emprunté quels livres (emprunt, retour, statistiques).

Ce sujet est un **fil rouge** : il vous fait progresser pas à pas des **listes** vers les **dictionnaires** et le format **JSON**, en réutilisant à chaque étape ce que vous avez construit à l'étape précédente.

Élément	Détail
Sujet	Gestion d'une bibliothèque de prêt (catalogue + emprunts)
Modalité	Travail en binôme
Langage	Python : listes, puis dictionnaires et JSON
Support de rendu	Un notebook Jupyter
Responsable de la SAE	B. LEMAIRE

● Point important à bien comprendre

Deux exigences traversent **toute** la SAE, de la première à la dernière partie :

1. **Chaque fonction doit être testée** par des tests unitaires ;
2. **Chaque fonction doit être comprise** par les deux membres du binôme, au point de pouvoir l'expliquer sans ordinateur (à l'oral et/ou lors d'un contrôle sur table).

Un code « qui marche » ne suffit pas : c'est un code que *vous* savez expliquer qui est évalué.

2. Calendrier

Jalon	Date	Ce qui se passe
Présentation de la SAE	Séance 7	Lancement du sujet, constitution des binômes, découverte du notebook.
Rendu de la SAE	18/11/2026	Dépôt du notebook complet et des fichiers associés (voir section 7).
Contrôle de SAE	19/11/2026, 13h45–15h45	Contrôle sur table, sans ordinateur : vérification de votre compréhension du code rendu.

● Attention

Le contrôle a lieu **le lendemain du rendu**. Ne comptez pas sur la dernière soirée pour « finir le code » : réservez les derniers jours à la **relecture** et à **l'entraînement à l'explication** de votre code (partie 8).

● Conseil

Fixez-vous des étapes intermédiaires : les parties Préliminaire, 1 et 2 constituent un premier bloc cohérent (tout en listes, tests fournis) ; terminez-le et faites-le valider en TD avant d'attaquer la transition vers les dictionnaires.

3. Travailler en binôme

Le travail est commun, mais l'évaluation de la compréhension est **individuelle dans les faits** : chacun doit pouvoir expliquer n'importe quelle fonction du notebook, même celle écrite par son binôme.

- **Alternez au clavier** : l'un écrit, l'autre relit et pose des questions, puis vous échangez les rôles.
- **Expliquez-vous mutuellement** chaque fonction terminée : si l'un ne sait pas l'expliquer, elle n'est pas terminée.
- Gardez **une seule version de référence** du notebook pour éviter de perdre du travail (nommez clairement les copies de sauvegarde, datez-les).
- Notez au fur et à mesure **pourquoi** vous avez fait un choix (structure de données, nom de fonction, cas particulier) : ces notes vous serviront directement pour la partie 8.

4. Déroulement du sujet, partie par partie

Le sujet progresse en deux temps, articulés autour d'un **changement de structure de données** : d'abord tout est modélisé avec des listes, puis vous migrez vers des dictionnaires et le format JSON.

Partie	Contenu	Tests fournis ?
Préliminaire	Modélisation du catalogue et des emprunts par des listes	Oui
1	Gestion du catalogue (ajout, suppression, recherche, fichier texte)	Oui
2	Gestion des emprunts (emprunt, retour, statistiques)	Oui
3	Transition listes → dictionnaires (fonctions de conversion)	Non
4	Sauvegarde et chargement au format JSON	Non
5	Réécriture des fonctionnalités sur dictionnaires	Non
6	Rédaction de vos propres tests unitaires	C'est vous qui les écrivez
7	Programme principal (menu interactif réutilisant les fonctions précédentes)	—
8	Vérification de compréhension sans IA : trace d'exécution, débogage, justification de choix, entretien oral	—

● **Attention**

À partir de la partie 3, plus aucun test n'est fourni. Ce n'est pas un oubli : c'est un choix pédagogique volontaire. Vous devez désormais écrire vous-mêmes les tests de vos fonctions. Ne cherchez pas les tests « comme avant » : écrivez-les.

Partie préliminaire — Modéliser avec des listes

Ce qu'on vous demande : représenter le catalogue et les emprunts à l'aide de listes, conformément à la modélisation imposée par le sujet. **Tests fournis**

- Lisez attentivement la spécification avant d'écrire la moindre ligne.
- Exécutez les tests fournis dès la première fonction : ils vous confirment que vous respectez la spécification.

Partie 1 — Gestion du catalogue

Ce qu'on vous demande : les fonctionnalités d'ajout, de suppression et de recherche de livres, ainsi que l'enregistrement et la relecture du catalogue dans un fichier texte. **Tests fournis**

Partie 2 — Gestion des emprunts

Ce qu'on vous demande : les fonctionnalités d'emprunt et de retour d'un livre, puis le calcul de statistiques. **Tests fournis**

● **Conseil**

Faites passer les tests fournis au fur et à mesure, fonction par fonction, plutôt qu'en une seule fois à la fin de la partie.

Partie 3 — Transition des listes vers les dictionnaires

Ce qu'on vous demande : les fonctions de conversion de vos données, de la représentation en listes vers la représentation en dictionnaires. **Aucun test fourni**

Partie 4 — Sauvegarde et chargement au format JSON

Ce qu'on vous demande : la sauvegarde et le chargement de vos données au format JSON. **Aucun test fourni**

Partie 5 — Réécriture des fonctionnalités sur dictionnaires

Ce qu'on vous demande : la réécriture des fonctionnalités des parties 1 et 2 sur la nouvelle structure de données.

Aucun test fourni

● À faire

Pour chaque fonction écrite dans les parties 3 à 5, écrivez vous-mêmes ses tests, avec la même rigueur que les tests qui vous étaient fournis auparavant.

Partie 6 — Vos propres tests unitaires

Ce qu'on vous demande : rédiger vous-mêmes les fonctions de test (`test_...`) de vos fonctions. C'est une partie évaluée en tant que telle.

Structure attendue d'un test

Un test est une fonction dont le nom commence par `test_`, qui appelle la fonction testée sur des données connues et vérifie le résultat avec `assert`. Si tout se passe bien, elle affiche `test ok`.

```
def test_nom_de_la_fonction():  
    assert ... == ...  
    ...  
    print("test ok")
```

Quels cas tester ?

- un ou plusieurs **cas généraux** ;
- les **cas limites** : structure vide, élément ou clé absent(e), etc.

● Conseil

Écrivez le test **avant ou juste après** la fonction, jamais « à la fin du projet ». Un test écrit tôt vous fait gagner du temps : il détecte immédiatement une régression quand vous modifiez une fonction.

Partie 7 — Programme principal

Ce qu'on vous demande : un menu interactif qui réutilise les fonctions écrites dans les parties précédentes.

- Réutilisez vos fonctions : le programme principal n'a pas vocation à les réécrire.
- **Robustesse** : votre programme doit réagir correctement face à une utilisation incorrecte, sans jamais s'interrompre sur une erreur.

Partie 8 — Vérification de la compréhension (sans IA)

Cette partie ne demande **aucun nouveau code**. Elle vérifie que vous maîtrisez réellement ce que vous avez rendu, selon quatre axes :

Exercice	Ce que vous devez savoir faire
Trace d'exécution	Suivre à la main l'exécution d'une fonction, ligne par ligne, en notant l'évolution des variables.
Débogage	Trouver et corriger l'erreur dans un morceau de code fourni, et expliquer pourquoi il était faux.
Justification de choix	Expliquer pourquoi vous avez choisi telle structure, telle boucle, tel découpage en fonctions.
Entretien oral	Présenter et défendre une fonction de votre notebook choisie par l'enseignant.

Méthode : réaliser une trace d'exécution

On dessine un tableau avec une colonne par variable et une ligne par passage dans la boucle :

```
total = 0
for n in [4, 1, 5]:
    total = total + n
```

Étape	n	total
Avant la boucle	—	0
Tour 1	4	4
Tour 2	1	5
Tour 3	5	10

● Conseil

Entraînez-vous en binôme : l'un choisit une fonction du notebook et un jeu de données, l'autre fait la trace sur papier, puis vous vérifiez en exécutant. Faites-le régulièrement, pas seulement la veille du contrôle.

5. Le langage attendu

Le sujet s'appuie **uniquement** sur les notions vues en cours : boucles `for` / `while`, conditions `if`, listes et dictionnaires de base, fonctions, lecture/écriture de fichiers, et le module `json` pour la partie 4.

● Attention

Les éléments **hors programme** sont à proscrire : imports de bibliothèques comme `collections.Counter`, compréhensions de liste ou de dictionnaire, méthodes avancées non vues en cours. Leur présence sera relevée : c'est un indice fréquent de code recopié ou généré sans compréhension (la question 38 du sujet traite explicitement ce cas).

Tout ce que demande le sujet peut être réalisé avec les notions vues en cours. En cas de doute sur une syntaxe, posez la question en TD.

6. Modalités d'évaluation

Critère	Poids	Ce qui est regardé
Décomposition en fonctions	15 %	Fonctions courtes, une responsabilité chacune, réutilisées plutôt que dupliquées.
Qualité du code et lisibilité	15 %	Noms de variables et de fonctions explicites, indentation propre, commentaires utiles.
Tests et documentation (fournis + rédigés)	15 %	Tests fournis qui passent, tests rédigés pertinents (cas limites), docstrings présentes.
Respect des spécifications (listes puis dictionnaires/JSON)	15 %	Structures de données et comportements conformes au sujet.
Programme principal et robustesse	15 %	Menu fonctionnel, saisies incorrectes gérées, aucun plantage.
Compréhension et défense du code (partie 8)	25 %	Traces, débogage, justifications, entretien oral.

● Point important à bien comprendre

Un binôme incapable d'expliquer, à l'oral et/ou à l'écrit sans ordinateur, une fonction qu'il a rendue voit **la note de cette fonction remise en cause**, quel que soit son fonctionnement apparent.

La compréhension représente à elle seule **un quart de la note** : ne la sous-estimez pas en pensant que « le code qui marche suffit ».

7. Livrables attendus

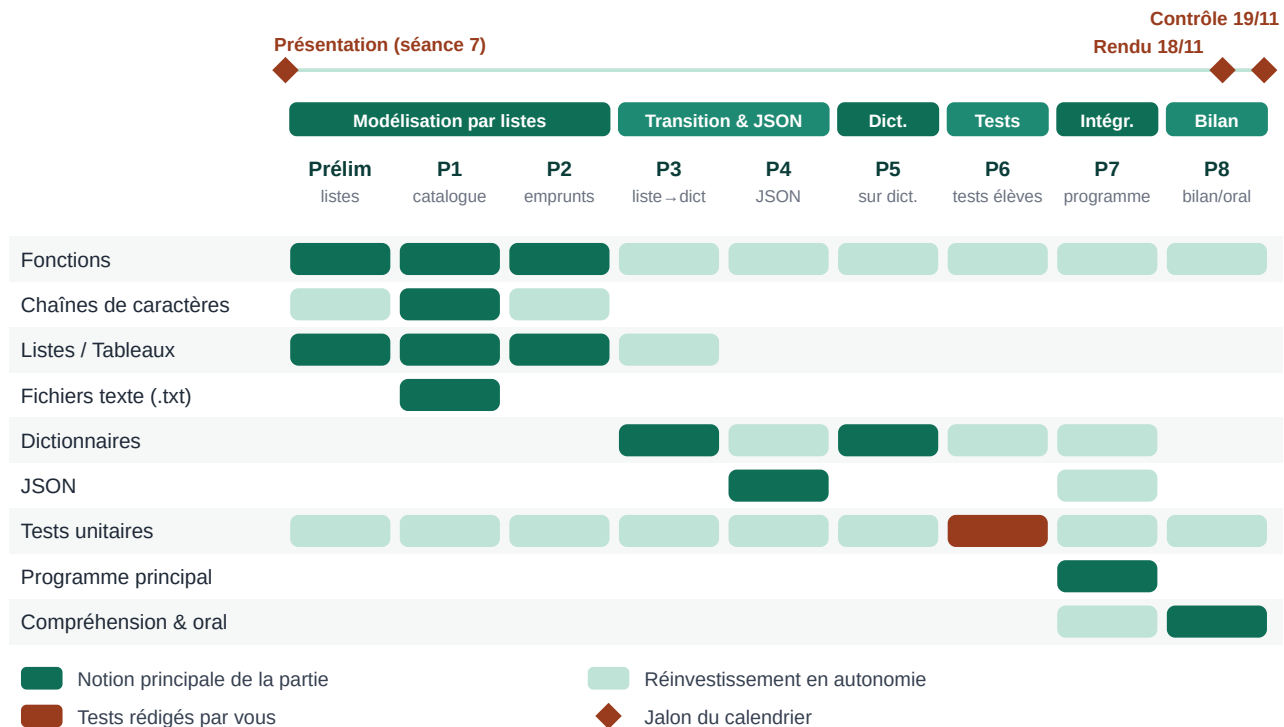
✓	Livable
☐	Notebook Jupyter complet (version listes et version dictionnaires)
☐	Tests unitaires : ceux fournis + ceux que vous avez rédigés (partie 6)
☐	Réponses rédigées de la partie 8
☐	Au moins un fichier de catalogue d'exemple, au format texte et au format JSON
☐	Une docstring sur toutes les fonctions
☐	Un code lisible, commenté lorsque c'est nécessaire

Ce que doit contenir une docstring

- ce que fait la fonction, en une phrase ;
- la description de chaque paramètre ;
- ce que la fonction renvoie.

8. Vue d'ensemble : notions mobilisées

La frise situe, pour chaque grande notion, la ou les parties où elle est **principalement travaillée** (teinte pleine) et celles où vous la **réinvestissez en autonomie** (teinte atténuée).



Déroulement de la SAE 1.01 et notions mobilisées

9. Questions fréquentes

Q Je ne trouve pas les tests de la partie 3, où sont-ils ?

Il n'y en a pas, volontairement : à partir de la partie 3, c'est à vous de les écrire (voir partie 6).

Q Mon code fonctionne, pourquoi devrais-je préparer la partie 8 ?

Parce qu'elle compte pour 25 % de la note et qu'une fonction que vous ne savez pas expliquer peut perdre ses points, même si elle fonctionne.

Q Puis-je utiliser une fonction ou une syntaxe trouvée sur Internet ?

Restez dans les notions vues en cours. Un élément hors programme sera relevé, et vous devrez de toute façon savoir l'expliquer et le réécrire sur table.

Q Faut-il des docstrings et des tests dès la partie 1 ?

Oui. Ils sont vérifiés au fil des séances de TD, pas seulement au moment du rendu.

Q Mon binôme a écrit une fonction que je ne comprends pas. Que faire ?

Demandez-lui de vous l'expliquer, puis réexpliquez-la-lui à votre tour. Tant que vous n'y arrivez pas tous les deux, la fonction n'est pas terminée.

Avant de rendre : tous les tests (fournis et rédigés) affichent `test ok` · chaque fonction a sa docstring · aucun élément hors programme · les fichiers d'exemple texte et JSON sont joints · les deux membres du binôme savent expliquer chaque fonction, sur papier, sans ordinateur.